FREE PROMPT PACK · CLAUDE CODE · JEV

# Give Your Jarvis *Reflexes* with Jev

Twelve copy-paste prompts in two parts. Part One builds the Jarvis from my videos: a galaxy of your own notes, a brain that answers from them, a voice both ways, a butler, and a preflight that keeps it honest. Part Two gives him reflexes: a 200-millisecond decision layer that knows where a question goes, whether you finished the sentence, whether you were even talking to him, whether your notes actually hold the answer, and whether he is about to claim something he never did. Every decision lands on screen as a bar you can read.

ZUBAIR TRABZADA

AI WORKSHOP

SEPTEMBER 2026

# How to use this pack

**What you need:** Claude Code installed and logged in, Google Chrome (the microphone and speech need it), a folder of markdown notes, an Anthropic API key from console.anthropic.com for the brain, and for Part Two a free key from typesafe.ai (a question costs a few thousandths of a cent; five dollars of credit lasts months).

**Set up in one line.** Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`. Then paste Prompt 01.

1 **Part Two assumes Part One.**
The reflex prompts bolt onto the server and viewer that Prompt 01 creates. If you already built Jarvis from an earlier pack, skip straight to Prompt 06; the prompts refer to "the server" and "the viewer" and Claude Code will find them.

2 **Paste each prompt whole, unedited, the first time.**
Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is where the reliability comes from. Run it as written before you customize.

3 **Never paste an API key into a chat window.**
Prompts 01 and 06 create placeholders in a config file. You type the real keys into that file yourself, in your own editor. Treat anything pasted into any chat as exposed. This is the one rule with no exceptions.

4 **Jev advises. Your code decides.**
Every threshold in Part Two is a named constant at the top of one file, with a comment. The model returns a probability; the code owns the permission. Every prompt says this, and every prompt has it verified. Keep it that way when you extend the build.

5 **Direct it with feelings, not specifications.**
After the first build, say "he jumps in before I finish," "he answered while I was talking to the camera," "the strip disappears too fast." Describing the problem beats prescribing the fix, because Claude Code can read the log and find the cause, and you can only guess at it.

# What Jev is, honestly

Jev is a decision model from TypeSafe AI. It never writes a sentence. You send it a piece of text plus typed questions, and it sends back a probability for each one, all at once, in about a fifth of a second. That is a different animal from the model that writes your answers, and the whole trick of this pack is using both: the big brain thinks, the small one has reflexes.

| | A CHAT MODEL (THE BRAIN) | JEV (THE REFLEX) |
|---|---|---|
| What it does | Writes prose, code, plans, jokes. | **Answers typed questions** with a probability: yes or no, a choice from a list, a score on a scale. |
| Speed | A few seconds. | **170 to 330 milliseconds** in my Jarvis, measured over a full day. |
| Cost | Cents per answer. | $0.042 per million input tokens. A question in my Jarvis costs **six-thousandths of a cent**. A day of building and talking came to two cents. |
| What it cannot do | Know when it is bluffing. | **Chat, explain, or write.** And it can be wrong: a probability is a judgment, which is exactly why every threshold lives in your code, never in the model. |
| Where it shines | Open-ended reasoning. | **The thousand small calls** an assistant makes before it speaks: which lane, did he finish, was that for me, does the vault have it, is this answer backed by a receipt. |

**What is not the model.** The strip, the gates, the judge's correction line, filming mode and the governor are mechanics you build in Part Two; they would work with any decision model. Jev supplies the numbers. The design, and the honesty, are yours.

# Part One · the foundation

Five prompts that build the Jarvis from the videos: a 3D galaxy of your own notes, a brain that answers only from them, a voice in both directions, the camera dive to the source note, a personality, and a preflight harness that tells you the truth about what works.

These are the five prompts that everything in Part Two depends on. The earlier packs in this same folder (Build Your Own Jarvis) add memory by voice, sight, screen watch and a brain swap; none of those are required for the reflexes, so add them whenever you like.

PROMPT 01 · THE FOUNDATION

# The Galaxy and the Brain

*Best for: the first paste. One prompt builds the whole foundation and starts it.*

```
Build me a 3D knowledge galaxy from my markdown notes, and give it a brain I can talk to. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER, or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER: build.py, Python 3, standard library only, no pip installs. Scan every .md file and write viewer/graph-data.js containing const GRAPH = {nodes: [...], links: [...]}. Each node gets a label from the filename, its parent folder as a group, and roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share [[wikilinks]]. CRITICAL: give every node a numeric id equal to its index in the nodes array. Later prompts look nodes up by index.

THE VIEWER: viewer/index.html, single page, using 3d-force-graph from a CDN. No npm, no build step, no framework. Cinematic: black space, a starfield, nodes glowing and colour-coded by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with that note's excerpt.

THE SERVER: server.py, Python standard library, port 4700, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/, containing {"api_key": "PUT-YOUR-KEY-HERE", "model": "claude-fable-5-1"}. Do not ask me for my key and do not put it anywhere the browser can reach.

THE BRAIN: add POST /chat to server.py. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and call the Anthropic Messages API with the model from config.json. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return {"answer": "...", "nodes": [array indexes of the notes you used]}. Keep a short conversation history server-side so follow-up questions work. Add a clean input bar at the bottom of the viewer that shows the answer on screen.

FINISH PROPERLY: start the server, confirm the page loads and the graph renders, and confirm /chat returns a well-formed response with a placeholder-key error rather than a crash. Then tell me exactly what to paste into config.json, and give me three example questions based on my actual note titles.
```

**Make it yours:** if you would rather not buy an API key, add one line: "instead of the Anthropic API, make /chat shell out to claude -p so it runs on my Claude Code subscription." Slower, but free.

PROMPT 02 · SPEECH

# The Voice, both directions

*Best for: the moment it stops being a website and starts being an assistant.*

```
Give it a voice, both directions, using only what the browser already has. No paid speech services.

SPEAKING: every answer is spoken aloud with the Web Speech API (speechSynthesis). Prefer a British English voice when the system has one. Browsers block audio until the user has interacted with the page, so make sure the first spoken line reliably fires after my first click, not before.

LISTENING: add a microphone button using webkitSpeechRecognition. I click it, I speak, the transcript goes through the same /chat flow as typing. Deliver the interim (partial) transcripts to a hook on the window as they arrive, because a later prompt will read them while I am still talking.

THE PART EVERYONE GETS WRONG: speech recognition finalizes a phrase when I pause, and I pause in the middle of sentences. So do NOT dispatch on the first final result. Buffer it, and put the wait in a single named constant at the top of the file called FINISH_MS, set to 900. If more speech arrives before that timer fires, append it and restart the timer. The pause that actually ends my thought sends the whole combined sentence. Short interrupt words like "stop" and "wait" must bypass the buffer entirely and fire instantly.

THE MUTE LAW: if the page URL contains ?mute=1, nothing in that tab ever speaks. Put the check inside the ONE function that speaks, so nothing can route around it. Every test tab from now on opens with ?mute=1, so I never hear the assistant from a background tab.

STATUS: a small status line that shows listening, thinking, and speaking states, so I always know which of us the machine thinks is talking.

VERIFY: then give me a test plan: exactly what to click, what to say, and what I should see and hear at each step, including one deliberate mid-sentence pause to prove the finish window works, and one ?mute=1 tab to prove the silence.
```

**Make it yours:** FINISH_MS is the single most personal number in this build. Too low and it cuts you off mid-thought. Too high and it feels sluggish. Start at 900, then tell Claude Code to raise or lower it after a day of real use. Do not guess at it in one sitting.

PROMPT 03 · THE DEMO MOMENT

# Make it prove where the answer came from

*Best for: the ten seconds of this build that make people lean toward the screen.*

```
When it answers from a note, I want the galaxy to prove it.

/chat already returns a nodes array of the indexes it used. Use it:

- Fly the camera to the top source node, light it up along with its direct neighbours, and open its side panel, so I can see exactly which note the answer came from while it is still speaking.
- If the answer drew on many notes, do not fly anywhere. Light the whole cluster instead. Flying to one arbitrary node out of six is a lie about where the answer came from.
- Never read the note aloud. It is on screen. The spoken answer stays short and the screen carries the detail.
- Small talk must not drag the camera around. If I say "good morning" or make a joke, the graph holds still. Only actual questions about my notes move the camera.

That last rule is the one that separates a demo from a toy, so implement it deliberately rather than as an afterthought: decide whether the question was about my notes before you decide whether to move the camera. Put that decision in ONE function, because a later prompt will hand it a much better opinion.

ONE MORE LAW: nothing except the answer path may move the camera or change which nodes are lit. Any other feature that wants to draw attention to a note may change its SIZE, never its focus. I learned this the hard way.

Then show me: give me one question that should fly to a single note, one that should light a cluster, and one that should not move the camera at all.
```

**Make it yours:** this is the shot worth recording. Ask something whose answer lives in one very specific note, and film the camera dive. It reads on video far better than any feature list.

PROMPT 04 · CHARACTER

# The Butler

*Best for: turning a question box into something people actually want to talk to.*

```
Give it a personality. Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:

- Address me as "sir", occasionally, not in every sentence. Over-using it is the difference between charming and grating.
- Answer in ONE witty sentence plus the facts. Never recite the note back to me, it is already on my screen.
- One genuinely funny line beats three bland ones. If nothing funny is available, be brief instead of forcing it.
- Handle small talk without dragging the camera around the graph.
- When my notes genuinely do not cover something, say so plainly and with a little dignity. Never invent a source, never pad, and never pretend a related note is the answer.
- Never claim an action was performed unless the code actually performed it and handed you a receipt. Offering to do something is fine. Saying "done" when nothing happened is the one unforgivable sin.

BOOT GREETING: when the page loads, it greets me with the real indexed note count, for example "Good evening, sir. 485 notes indexed, all present and accounted for." Read the actual number from the graph data, do not hardcode it. Make the time of day correct too.

WAKE WORD: a hands-free mode where the microphone stays open and the assistant responds only when a sentence starts with its name. Strip the name before the sentence goes to /chat. Keep the wake-word test in ONE regular expression near the top of the file, because the recogniser will mishear the name and a later prompt will add the variants.

Put the entire persona in ONE clearly commented block at the top of server.py, so I can rewrite the character later without hunting through the file.

Then give me five things to say to it that will show off the personality, including one question it definitely cannot answer from my notes.
```

**Make it yours:** the butler is just the default. Swap the persona block for a terse mission-control operator, a sarcastic friend, or a patient teacher. Keeping it in one block is what makes that a thirty-second change.

PROMPT 05 · THE ONE THAT MATTERS

# The Preflight

*Best for: the difference between a weekend demo and something you actually rely on.*

```
Build me a preflight harness. This is the most important prompt in this pack, so take your time with it.

Write preflight.py that executes every LIVE chain in this project end to end and prints a verdict. Not unit tests. Not
mocks. Real calls against the running system, because the failures that hurt are the ones where every unit test passes
and a real chain is dead.

It must check, in order, and print a tick or a cross for each:
- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- /chat returns a well-formed answer to a real question, with a nodes array.
- The API key in config.json is actually valid, by making one real minimal call.
- The model named in config.json is one the key can actually reach.
- The files the browser is actually being SERVED match the files on disk. A stale served file is the single most common
cause of "I fixed it but nothing changed".
- config.json is not reachable from the browser. This one must fail loudly.
- A ?mute=1 tab really is silent: the one speaking function refuses when the flag is set.

Finish with a single summary line: N pass, N fail, N warn. Exit non-zero on any failure.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not
describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste the summary line. "Done" means
preflight passed, not that the code looks right. Every prompt in Part Two will add checks to this file.
```

**Make it yours:** add a check every time something breaks on you. A harness that grows one check per real incident becomes the most valuable file in the project within a month. Mine has 57 checks and every one of them was earned.

# Part Two · the reflexes

Seven prompts that put Jev in front of everything Jarvis hears and says. The first one is the bay: the client, the laws, the log. Then the five questions asked of every sentence, shown live on a strip. Then the vault check, the judge, filming mode, the desk, and the drill that proves it all before you trust it with a microphone.

| | |
|---|---|
| **Lane** | Where does this go: my notes, my calendar, small talk, a job, the reflex itself? Trusted at 70 percent, otherwise the brain decides. |
| **Complete** | Did he finish the sentence, or is it dangling on "and"? High enough, and Jarvis may act before the microphone has even finalized. |
| **Addressed** | Was that aimed at the assistant, or at a camera, another person, thinking out loud? This is what makes filming mode possible. |
| **Stakes** | Would acting send, call, delete, post, or spend? Above the gate, he confirms first. |
| **Vault has it** | Do the notes actually contain the answer? Found, the notes light. Absent, he says "not in the vault" instead of bluffing. |

Two laws hold across all seven prompts. Every gate is a named constant with a comment, retuned only from logged distributions. And every call has a deadline: if Jev is slow or down, Jarvis falls back to the rule he had before, the same second. The reflex may make him smarter; it may never make him slower or silent.

PROMPT 06 · THE BAY

# The Reflex Bay

*Best for: the first paste of Part Two. The client, the laws, and the log that every later prompt builds on.*

```
Give my Jarvis a reflex layer with Jev. Jev is TypeSafe AI's decision model: it never writes text, it returns a probability for every typed question you send, in about 200 milliseconds. Read https://docs.typesafe.ai/llms.txt before you write a line, and trust the live docs over anything you remember.

THE KEY: add "typesafe": {"api_key": "PUT-YOUR-TYPESAFE-KEY-HERE", "model": "jev-latest", "enabled": true} to config.json. I will paste the real key into the file myself. Never ask me for it, never print it, never let the browser see it. While the placeholder is there, everything in this layer idles quietly and Jarvis behaves exactly as before.

THE BAY: reflex.py, Python standard library only, imported by server.py. One function ask(state, questions, purpose, deadline) that POSTs to https://api.typesafe.ai/v1/systemone with the model from config, measures the round trip, counts input tokens, and prices them at $0.042 per million. Three builders: noul(instructions, criteria), choice(instructions, criteria_map), score(instructions, levels). ask() NEVER raises: on a timeout past the deadline, an HTTP error, or a missing key it returns None, and every caller falls back to whatever rule it used before. The reflex may make Jarvis smarter; it may never make him slower or silent.

THE LAWS: every threshold this layer will ever use is a NAMED CONSTANT at the top of reflex.py with a one-line comment, never a number buried in a function. Jev advises; the code decides. Every call is logged as one JSON line to jarvis-reflex.jsonl (owner-only file permissions) with the purpose, the model, the milliseconds, the tokens, the text it judged, and the FULL answer distributions, so thresholds can be retuned from real data later without re-running anything.

THE DOORS: GET /reflex/diag returns whether the key is present, the model, today's call count, tokens, cost, the p50 and p95 latency, and every gate by name. GET /reflex/log?n=5 returns the last five log lines. Rebuild today's tally from the log at boot, so a restart never forgets the morning.

VERIFY: with the placeholder key, start the server and show me /reflex/diag reporting enabled false with no traceback. When I say the key is in, run one real call with three questions about the sentence "book me a table for four at seven tonight" and paste the raw answers, the milliseconds, and the cost. Do not describe it. Run it.
```

**Make it yours:** pin the model to the exact version you tuned on ("jev-1.13.0" as I write this) the moment your gates start to matter. Aliases move; a gate tuned on one version is a guess on the next.

PROMPT 07 · THE STRIP

# The Prepass and the Strip

*Best for: the five questions on every sentence, shown live while you are still talking.*

```
Put the reflex in front of every sentence I say to Jarvis, and show me the decisions.

THE PREPASS: one Jev call per utterance, BEFORE the brain sees it. State: the utterance, plus a short list of my own
note titles (Jev could never know my product names otherwise). Five questions: lane (a choice over vault question /
chat / calendar and mail / agent task / reflex / none, each described in a full sentence; "vault question" includes a
request to open or pull up a note by name), complete (noul: has he finished a complete request, or does it end on a
connector like "for", "and", "the"), addressed (noul: aimed at the assistant, not narration or another person), stakes
(noul: would acting send, call, delete, post, or spend), urgent (noul). Return a decision built from named gates:
LANE_ACT 0.70, LANE_CLARIFY 0.50, COMPLETE_FIRE 0.85, ADDRESSED_ROOM 0.80, STAKES_CONFIRM 0.50.

THE STRIP: a panel above the ask bar with one row per question: the question in plain words, a bar for the probability,
a thin tick at the gate, the number, and a verdict chip (act, ask, hold, clear, confirm). A pill in the corner shows
the model, the question count, the milliseconds, and the cost of THIS turn. Flash a row amber when a value crosses its
gate, with a small "62% to 97%" receipt. The rows stay on screen until I fold them with a plain fold/open button, and
the next question opens them again. Never auto-hide anything I might be explaining to a camera. Nothing on this panel
is decoration: every number is Jev's answer for this turn.

STREAMING: send the partial transcripts to the prepass as they arrive, debounced, so the bars settle while I am still
talking. Early dispatch (acting before the recogniser finalizes) is allowed only when complete and lane both clear
their gates, stakes is below its gate, AND the partial text has been unchanged for EARLY_STABLE_MS (600). Only silence
can tell a finished sentence from a breath. If the final transcript grows past 1.5 times the words of the one that
fired, ask it again; never swallow the rest of a sentence.

VERIFY: type "what is on my calendar today" and "pull up my notes on pricing" and paste both decision objects with
their milliseconds. Then add a probe URL, ?reflexprobe=1&mute=1, that silently runs ten checks (the strip builds, the
chips, the pause guard holding a prefix while more words arrive, hold mode keeping the rows open, one live prepass) and
writes REFLEXPROBE n/10 into the page title.
```

**Make it yours:** "complete" cannot tell a grammatical prefix from the full sentence: "what is Acme" scores 0.92 as the start of "what is Acme and what does it charge". Never lower EARLY_STABLE_MS to chase speed. The pause is the signal.

PROMPT 08 · THE VAULT

# The Vault Check

*Best for: an assistant that says "not in the vault" instead of bluffing from a note that merely sounds related.*

```
Make Jarvis know whether my notes actually hold the answer, before the brain speaks.

THE RECALL: when the lane is a vault question, take the 40 best keyword matches from my notes and send ONE Jev call with the question and the 40 titles plus short excerpts. Ask: has (noul: does at least one note DIRECTLY contain the answer; true means it states the specific fact, number, decision or plan; false means the notes are only on a similar topic), names (noul: is the question asking to open or pull up a note by name rather than asking a fact), ext (score over five levels: none, only the top note, the top three, half the list, most of the list), and one noul per note: would a correct answer draw on this note. Verdict from named gates: FOUND 0.70, ABSENT 0.35, NOTE_KEEP 0.40, NAMED_OPEN 0.70. Found: keep the notes above NOTE_KEEP, capped by ext. Named: the top note is the answer. Absent: keep nothing. Between: unsure.

THE BRAIN OBEYS: found, and the system prompt tells the model these notes were verified to contain the answer and it must answer from them in one line. Absent, and the model is told in the prompt that the notes contain NOTHING on this and must say so plainly instead of improvising. Unsure, and the brain decides as it does today.

ON SCREEN: a sixth row on the strip, "Vault has it", with the probability and a chip reading found, unsure, dark, or named. Nodes SWELL in the galaxy in proportion to their probability while I am still talking, and shrink back after 30 seconds. The reflex never moves the camera and never changes which nodes are lit; the answer path already does that.

SPEECH-PROOF NAMES: the recogniser mangles names. Before the recall, phonetically match every one-to-three-word gram of the question against note titles (fold soft c, d and t, v and f), put those notes first, and rewrite the misheard words to the vault's own name. Strip the wake word and its misheard variants first. Only real names may rewrite a question, never generic titles, and a longer match always beats a word inside it.

VERIFY: ask one question a specific note answers (expect found, that note swollen), one that names a note (expect named), and one about the weather (expect absent, and the spoken words "not in the vault"). Paste the three verdicts. Add both cases to preflight.
```

**Make it yours:** when he "cannot find a note that exists," read the log before you touch the vault. Nine times out of ten the recogniser wrote "school" for "Skool" and the correction is one phonetic rule away.

PROMPT 09 · THE JUDGE

# The Judge

*Best for: an assistant that cannot say "done" when nothing happened.*

```
Before any spoken answer reaches my ears, put it in front of a judge.

THE CALL: after the brain answers and before the text is spoken, one Jev call with the question, the answer, and the list of tool receipts from this turn (what was actually sent, booked, posted or scheduled; "none" if nothing). Two nouls: claims (does the answer state the assistant has just PERFORMED an action with an outside effect this turn: sent, called, booked, posted, paid, deleted; "I've sent it" is a claim, an offer or a draft is not) and backed (is every claimed action supported by a receipt).

THE LAW: CLAIM 0.70 and BACKED 0.40 as named constants. If claims is at or above CLAIM, backed is below BACKED, and the receipt list is empty, the answer is BLOCKED. Replace it with one honest line spoken in character: "I nearly told you that was done, sir, and there is no receipt for it. Nothing has been sent, booked, or posted this turn. Shall I actually do it?" Log the blocked original with both probabilities.

ON SCREEN: a card with the struck-through original, the two probabilities against their gates, and the corrected line beneath. The banner reads "blocked, claim without receipt, corrected".

THE DEMO: a good brain refuses to fabricate on request, so give me a voice route, "show me the judge", that runs a canned fabricated line through the REAL judge, labelled "demo, a fabricated claim" on the card, so I can show the mechanism on camera without lying to anyone.

NEVER: the judge never touches an answer that carries receipts, and a judge timeout means the answer goes through unchanged. Latency budget 0.8 seconds.

VERIFY: run the judge on three answers: a true report with a receipt, an offer ("shall I send it?"), and a fabricated "done, sent" with no receipt. Paste the three verdicts with milliseconds. Only the third may block. Add it to preflight.
```

**Make it yours:** the judge is only as good as the receipts. Every tool your Jarvis can run must return a one-line receipt of what it actually did, and that list, not the brain's memory, is what "backed" is checked against.

PROMPT 10 · FILMING MODE

# Talk to the camera, not to him

*Best for: filming a video with Jarvis in the room and the wake word off.*

```
Let me film with Jarvis in the room. In filming mode the wake word is off, and he answers only when I am clearly talking to him.

THE MODE: "we're filming" turns it on, "filming mode off" turns it off; both must work with or without his name, as hard rules in code that never depend on Jev. Persist the flag server-side so a reload keeps it. Show it plainly: the strip header reads "filming mode" and a pulsing dot marks the ear.

THE GATE: every final transcript with no wake word goes to the prepass with recording_mode true, which adds one question: narrating (noul: is the speaker talking to a camera or an audience, explaining or presenting). He answers only if addressed is at or above ADDRESSED_FILMING and narrating is below NARRATING_QUIET. Start ADDRESSED_FILMING at 0.70 and NARRATING_QUIET at 0.60. When I measured this, narration to camera scored 0.06 to 0.18, and sentences aimed at him without his name scored 0.76 to 0.98, so 0.70 answers every ask and no narration. The lane is advisory here: a weather question aimed at him with a shaky lane still gets answered.

ON SCREEN: when he stays quiet the banner says why: "narrating, addressed 17%, gate 70%, staying quiet". When he answers: "addressed 78%, answering". While filming, the strip never folds on its own.

THE DRILL: write a script that posts twelve sentences to the prepass in filming mode: six that talk ABOUT him in the third person to an audience, six that ask him something without his name. Print addressed and narrating for each, then report how many narration lines would have been answered and how many asks would have been missed at the current gates. If either count is not zero, propose the gate from the distribution, never from a guess.

VERIFY: run the drill and paste the table. Add two checks to preflight: narration stays quiet, a no-name ask is answered.
```

**Make it yours:** the six narration lines should be things you actually say on camera, including his name in the third person ("Jarvis is going to answer that"). That is the sentence that fools a naive wake word and does not fool this.

PROMPT 11 · THE DESK

# Triage and the Shadow Governor

*Best for: an inbox scored in a blink, and a governor that earns your trust in the log before it gets a voice.*

```
Every card that lands on Jarvis's desk (a comment, a message, a mail, a tool report) gets scored in a blink, and the cards that break in with voice get chosen by a governor that learns in shadow before it is trusted.

TRIAGE: a voice route, "triage the desk". Send the open cards to Jev in batches of no more than 12 per call (at 40 the answers cross-talk between neighbours), in parallel. Per card: kind (a choice over question, lead, praise, complaint, spam, other, each described), reply (noul: needs a personal reply), urgent (noul: within the hour). Attention score in code: reply times 0.6 plus urgent times 0.4, times 1.25 for leads and complaints, times 0.15 for spam. Show a panel with every card and its kind chip, and speak the three that matter.

THE GOVERNOR: the function that decides whether a new card interrupts me by voice already has a ladder: money always breaks through, then focus or do-not-disturb, then quiet hours, then priority. Keep that ladder in code. Add seven atomic Jev questions per card: money, waiting (a real person is waiting on him), time (harm if it waits for the next work block), broken, lead, routine (a status report with nothing to do), noise (spam or engagement bait). Situational score in code: a base by priority plus weighted nouls (money .30, waiting .25, time .25, broken .20, lead .30, routine minus .35, noise minus .40). Gates: GOV_SPEAK 0.55, GOV_URGENT 0.75, GOV_MONEY 0.70.

SHADOW FIRST: GOVERNOR_MODE = "shadow" by default. The old ladder still decides, and Jev's verdict is computed in a background thread and logged beside it as AGREE or DIFF. Only "live" lets the score decide, with a 0.8 second deadline and the old rule as the fallback. "off" never asks.

RECEIPTS: write a script that scores every card in the desk history under normal hours and under a simulated focus session, then prints how many the old rule would speak, how many the governor would, the agreement, every disagreement with its seven numbers, and the distribution of each noul. Run it and paste the tables. Do not flip to live until the DIFF lines look right for a week.
```

**Make it yours:** on my desk the old rule spoke 200 of 200 cards and the governor spoke 46, and it was the noise question that saved it: without one, a "FREE AI tools, sub for sub" comment scored 0.87 because "FREE" read as money.

PROMPT 12 · THE PROOF

# The Field Drill

*Best for: proving all of it without a microphone, then watching it with one.*

```
Before I trust any of this with a microphone, prove it without one, then let me watch it with one.

THE DRILL: test_reflex_drill.py posts the exact sentences of a field pass to the live reflex doors: a note question, a question my notes cannot answer, two narration lines and two asks in filming mode, the reflex voice phrases, and one destructive ask. For the spoken questions, also post the partial prefixes the recogniser would stream, three words at a time. Print one line per call with the lane, its confidence, and every noul, marked with the gates it crossed (TRUST, FIRE, ADDR, CONFIRM). At the end, name every misfire in plain words: early dispatch on a prefix, a narration line that would be answered, an ask that would be missed, a lane that landed wrong. If it is clean, say so.

THE WATCHER: reflex_watch.py tails jarvis-reflex.jsonl and prints each call in English: the time, the purpose, the text judged, the numbers, the verdict, and for governor lines the old verdict beside Jev's. I will run it in a terminal while I speak.

THE PROBE: the ?reflexprobe=1&mute=1 page from Prompt 07 grows one silent check per feature: the vault heat, the judge blocking a claim without a receipt, the pause guard, hold mode. It writes REFLEXPROBE n/n into the page title, and n/n is the number I read before I say any of this works.

TUNING LAW: a gate moves only from a logged distribution, never from one bad take, and every move gets a one-line comment naming the numbers that justified it. When I report a misfire in plain words, read the log for that sentence first, then propose the one gate and the one number.

VERIFY: run the drill, open the probe, run preflight, and paste all three summary lines. Then give me the script for my first real-mic pass: the exact sentences to say, in order, and what the strip should show for each.
```

**Make it yours:** my first real pass found the recogniser writing "Travis" for "Jarvis", and the name correction turned it into a random note title. The drill never caught it, the log did. Keep both.

# Seven things I learned putting Jev inside Jarvis

1 **Gate on silence, not on "complete".**
A grammatical prefix scores as a finished sentence, because it is one. "What is Acme" read 0.92 on its way to "What is Acme and what does it charge." Only a real pause in the transcript can tell the two apart, so the pause guard is the gate and the noul is the hint.

2 **"Addressed" is the cleanest signal in the whole build.**
Across 24 sentences, talking to the camera never scored above 0.18, and talking to him without his name never scored below 0.76. That gap is what makes filming with the wake word off possible.

3 **Batches cross-talk past about twelve items.**
At 40 cards per call, a spam comment inherited "money" from the payment card next to it, and an invoice complaint read as praise. Eight to twelve per call keeps the answers clean; one per call is the accurate path when it matters.

4 **A wrong probability is a bug in your thresholds, not in Jev.**
"FREE" reads as money until you ask a noise question. A cold sponsorship pitch reads as promotion. The fix was never a prompt tweak; it was one more atomic question and a line in the log to prove it.

5 **A good brain will not lie for your demo.**
Asked to say "done, sent" with nothing sent, the brain refused every time ("not a liar who confirms emails that never left the building"). So the judge demo runs a canned line through the real judge, labelled as such. Honest on camera, and it still shows the mechanism.

6 **When he cannot find a note that exists, it is the microphone.**
"School" for "Skool", "side view" for a product name, "Travis" for "Jarvis". The vault was fine every time. Phonetic matching on note titles, and stripping the misheard wake word, fixed all of it, and the log is where you see it.

7 **The reflex advises with bars, banner and swell, and never touches the camera.**
The one time it lit notes on its own, the galaxy went dark a second before every answer. Focus belongs to the answer path. Everything else may only make a note bigger.

> **The receipt.** A full day of building, drilling and talking to him: 228 reflex calls, 439 thousand tokens, $0.018. Median 220 milliseconds. Every number on the strip in my videos is one of those calls.

# Where to go from here

Run Part One in an afternoon, live with the butler for a day, then run Part Two one prompt at a time and read the log after each. The strip will teach you more about how you actually talk to a machine than any tutorial can, because every bar is a real decision about a real sentence you said.

1 **Post what you built.**
The free community has a thread for every pack. A screenshot of your strip, with the numbers, is the best question you can ask.

2 **Tune from your own log, never from mine.**
The gates in this pack are where my numbers landed. Yours will land somewhere else, and the drill in Prompt 12 will tell you exactly where.

3 **Then give the reflex to everything else.**
Any assistant, agent or automation you run has the same thousand small calls to make. The bay in Prompt 06 is portable on purpose.

## Want the finished Jarvis with the reflexes already wired?

Inside the paid AI Workshop community you get the full build I run every day: the reflex layer, the strip, the judge, filming mode, the desk, the phone line, and a five-minute install with no code. Plus the classroom, the calls, and a team that helps when something breaks.

**Free community: skool.com/aiworkshop-lite**
**Full workshop: skool.com/aiworkshop**
**YouTube: Zubair Trabzada | AI Workshop**